Université de Strasbourg

École doctorale Mathématiques, sciences de l'information et de l'ingénieur | ED 269 Université de Strasbourg

UNIVERSITE DE STRASBOURG

ECOLE DOCTORALE MATHEMATIQUES SCIENCES DE L'INFORMATION ET DE L'INGENIEUR

RESUME DE LA THESE DE DOCTORAT

Discipline : Informatique

Spécialité (facultative) :

Présentée par : BATTISTON Ugo
(Nom Prénom du candidat)

Titre : Optimisation polyédrique automatique des noyaux Kokkos : un co-design bibliothèque-compilateur

Unité de Recherche : Laboratoire des sciences de l'ingénieur, de l'informatique et de l'imagerie (UMR 7357)
(N° et Nom de l'Unité)

Directeur de Thèse : CLAUSS Philippe - Professeur
(Nom Prénom - Grade)

Co-Directeur de Thèse (s'il y a lieu) : PÉRACHE Marc - Directeur de recherche
(Nom Prénom – Grade)

Localisation : Strasbourg

Thèse confidentielle : ☐ NON ☐ ~~OUI~~

Introduction

Les besoins croissants en puissance de calcul ont grandement fait évoluer les architectures, en commençant par l'augmentation de la fréquence des processeurs, puis, pour pallier les contraintes physiques, par la multiplication du nombre de coeurs de calcul. Par la suite, d'autres paradigmes de programmation sont apparus avec l'avènement des architectures hétérogènes, rendant le développement de plus en plus complexe. Toutefois, exploiter efficacement ces ressources demeure un défi majeur : le développement de codes scientifiques de haute performance devient particulièrement ardu pour les physiciens et mathématiciens, car il requiert désormais des connaissances approfondies en ingénierie logicielle pour le calcul haute performance.

Pour pallier cette complexité, la communauté scientifique a conçu des bibliothèques de portabilité des performances comme Kokkos. Ces bibliothèques permettent de déléguer la gestion de la complexité architecturale aux développeurs du framework. Elles offrent ainsi aux scientifiques la possibilité d'écrire un code source unique capable de s'exécuter sur de multiples architectures, en masquant les difficultés liées à la programmation hétérogène.

Dans cette thèse, nous allons nous intéresser à l'optimisation des nids de boucles au sein de Kokkos par l'utilisation du modèle polyédrique.

En effet, bien que ces bibliothèques facilitent grandement l'expression du parallélisme, elles n'effectuent pas d'optimisation agressive sur la structure même des nids de boucles, qui concentrent pourtant la grande majorité du temps d'exécution. D'un autre côté, le modèle polyédrique est une formalisation mathématique puissante permettant l'analyse et la transformation complexe de ces boucles (réordonnancement, tuilage, vectorisation).

Position du problème

Différents outils existent dans le monde de la compilation polyédrique pour appliquer automatiquement des transformations complexes à un code source. Une grande part de ces outils opèrent en source-à-source et se limitent souvent à un sous-ensemble du langage C. Par conséquent, pour l'analyse de code Kokkos, un framework C++ moderne utilisant intensivement la métaprogrammation pour abstraire le matériel, ces outils se révèlent inadaptés.

Une autre approche consiste à s'appuyer sur les représentations intermédiaires d'un compilateur, comme l'IR de LLVM, afin de s'affranchir des complexités liées au langage source. Toutefois, le code intermédiaire généré par Kokkos n'est pas immédiatement exploitable par ces outils : certaines informations structurelles cruciales sont perdues lors de la compilation en raison des mécanismes internes de la bibliothèque.

Ce fossé sémantique entre le code source et ce que le compilateur perçoit rend les optimisations polyédriques classiquement impossibles. Le défi consiste donc à réussir à faire bénéficier les noyaux de calcul exprimés avec Kokkos des optimisations agressives et mathématiquement prouvées du modèle polyédrique, sans pour autant sacrifier la productivité et la portabilité offertes par la bibliothèque.

Dans la première partie de nos travaux, nous détaillerons comment une approche de co-design entre la bibliothèque et le compilateur permet de résoudre ce problème. Dans un second temps, nous décrirons comment étendre cette approche à des schémas d'exécution plus complexes.

Un Co-Design Bibliothèque-Compilateur

Dans cette thèse, nous proposons une approche hybride reposant sur des modifications conjointes au sein du framework Kokkos et de l'optimiseur polyédrique LLVM Polly. Du côté de la bibliothèque, nous avons intégré des informations sémantiques de haut niveau directement dans le code source de Kokkos. Cette modification permet d'obtenir une représentation sémantique simplifiée au niveau de la représentation intermédiaire (IR).

En parallèle, nous avons étendu les capacités de Polly. Grâce à cette IR annotée simplifiée, et à des passes de transformation spécialement conçues pour analyser du code Kokkos, l'optimiseur est désormais capable d'identifier spécifiquement les constructions de Kokkos. En s'appuyant sur de nouvelles heuristiques, Polly peut ainsi extraire ces noyaux et reconstruire le modèle polyédrique.

Cette synergie crée un pont direct : la bibliothèque fournit les informations structurelles de haut niveau, et le compilateur polyédrique applique les transformations de boucles (comme le pavage ou la fusion) de manière automatique et optimale, avant de générer le code machine.

Schémas complexes et hétérogénéité

Une étape cruciale consistait à déterminer comment le modèle polyédrique, qui opère traditionnellement sur un noyau de calcul dans son ensemble, pouvait être appliqué aux noyaux Kokkos, ces derniers pouvant être découpés en de multiples exécutions de sous-noyaux. Nous avons exploité la métaprogrammation de Kokkos ainsi que notre capacité à manipuler la représentation intermédiaire afin de fournir à Polly une vision globale du noyau de calcul, lui permettant ainsi de l'optimiser dans son ensemble et non de manière locale.

Nous avons ensuite étendu notre outil pour qu'il soit capable de gérer des schémas d'exécution complexes, tels que les boucles triangulaires présentes dans de nombreux noyaux scientifiques, qui posent problème à Polly avec Kokkos comme code source.

De plus, la portabilité des performances exigeant de cibler des accélérateurs matériels, nous avons intégré des ordonnanceurs externes (tels que Pluto) et ajouté le support de la génération de code hétérogène pour GPU via PPCG. Ainsi, à partir d'un code Kokkos standard, notre chaîne de compilation est capable de générer automatiquement un code CUDA optimisé polyédriquement.

Conclusion

Dans cette thèse, nous avons apporté une nouvelle méthode d'optimisation hybride combinant la productivité des bibliothèques de portabilité des performances avec la puissance analytique du modèle polyédrique.

L'approche développée a été intégralement automatisée au sein d'une chaîne de compilation complète basée sur LLVM. L'évaluation expérimentale menée sur divers codes scientifiques a démontré que notre framework est capable de surpasser largement les performances des versions Kokkos standards.

Ces travaux valident la faisabilité et l'intérêt d'un co-design bibliothèque-compilateur, ouvrant ainsi la voie à des optimisations automatiques poussées pour les supercalculateurs.

Publication

Ugo Battiston, Philippe Clauss et Marc Pérache, *Unlocking Polyhedral Optimization for Kokkos Kernels: A Library–Compiler Co-Design*, ICA3PP 2026, 26th International Conference on Algorithms and Architectures for Parallel Processing, Lecture Notes in Computer Science (LNCS), Springer,, Exeter, UK, October 2026.